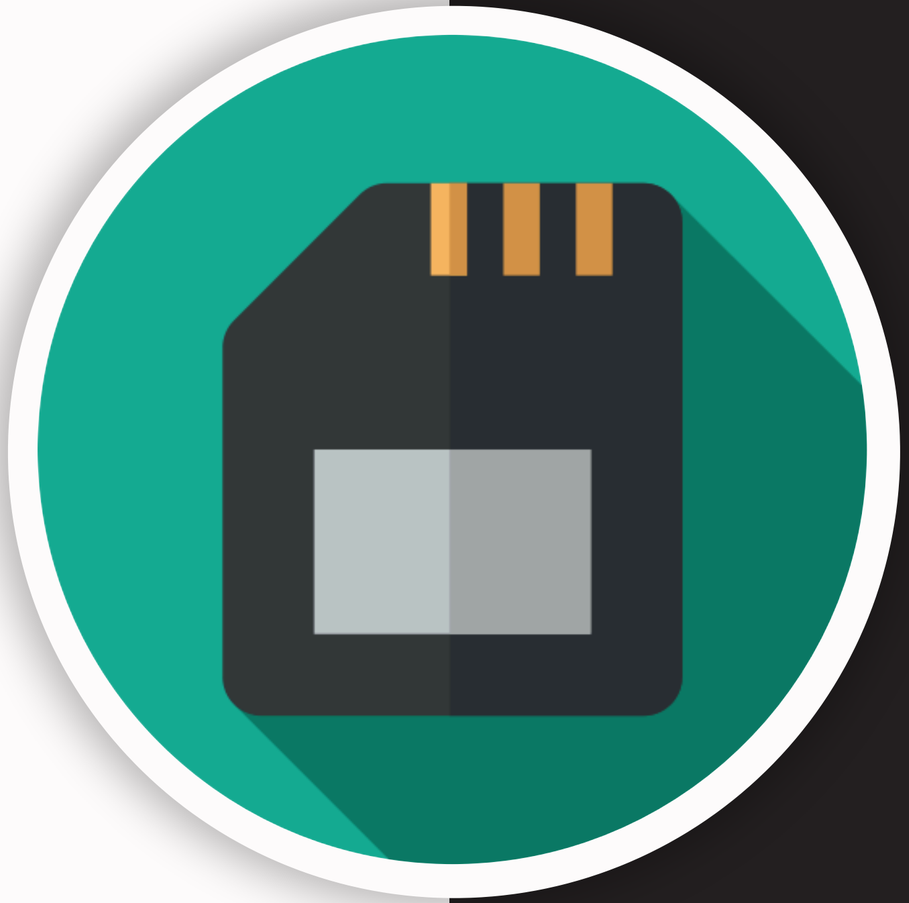
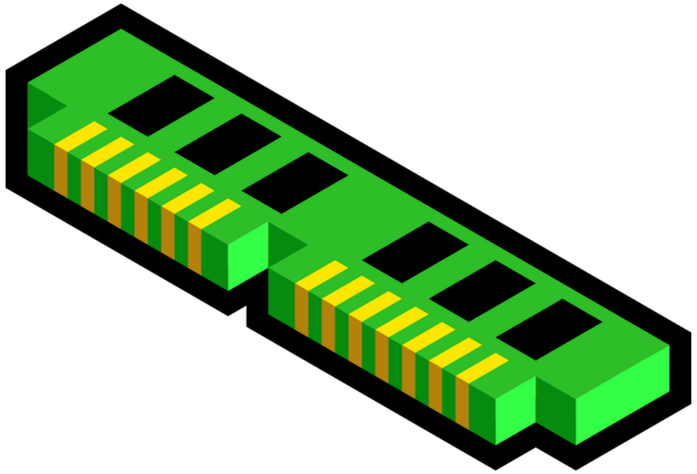
BELLEK YÖNETİM SORUNLARI



İçerik

- Boş Bellek Erişimi (Volkan ÖZDEMİR) **01**
- Bellek Sızıntısı (Volkan ÖZDEMİR) **02**
- Fragmentasyon (Volkan ÖZDEMİR) **03**
- Bellek Çakışması (Onur AKÇAY) **04**
- Yavaş Bellek Erişimi (Onur AKÇAY) **05**



Boş Bellek Erişimi

Sorunun Tanımı

Boş bellek erişimi, bir programın önceden ayrılmamış veya serbest bırakılmış bir bellek alanına erişmeye çalışmasıdır. Bu durum, sistemde hatalara yol açabilir ve uygulamanın çökmesine sebep olabilir. Özellikle büyük yazılımlarda bu hata, güvenlik açıklarına da yol açabilir.

Sorunun Çözümü

Bu sorunu çözmek için, bellek erişimi yapılan her yerde belleğin gerçekten ayrıldığından emin olunmalıdır. Programlama sürecinde bellek yönetimi kurallarına uyarak ve modern programlama dillerindeki bellek güvenliği özelliklerinden yararlanarak bu tür hataların önüne geçilebilir.

Boş Bellek Erişimi

Güvenlik Açıklı Kod

```
void testDanglingPointer() {
    int *ptr = (int*)malloc(sizeof(int));  // Bellek tahsis ediyoruz.

    if (ptr == NULL) {
        printf("Bellek tahsisi başarısız!\n");
        return;
    }

    *ptr = 42;  // Belleğe bir değer atıyoruz.
    printf("Belleğe atanan değer: %d\n", *ptr);

    free(ptr);  // Belleği serbest bırakıyoruz.

    printf("Serbest bırakıldıktan sonra ptr işaret ediyor: %d\n", *ptr);  // Boş belleğe erişim! Hata!
}

void main() {
    testDanglingPointer();
}
```

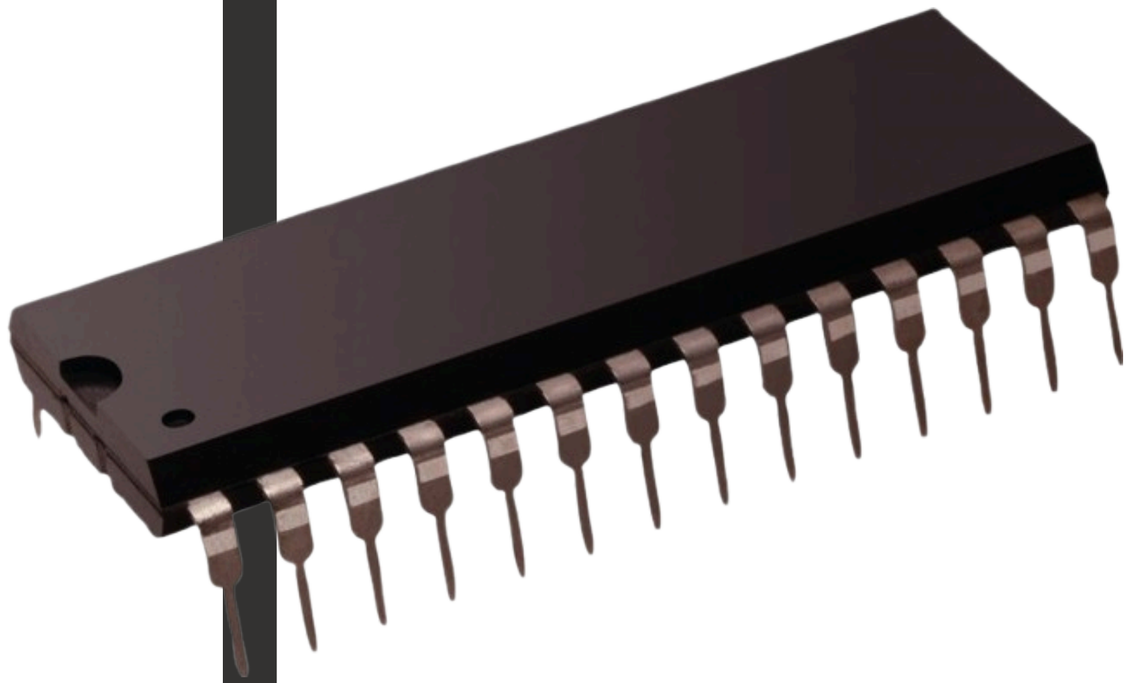
Bellek Sızıntısı

Sorunun Tanımı

Bellek sızıntısı, bir programın ihtiyacı olmayan verileri bellekte tutmaya devam etmesidir. Bu durum, sistem belleğinin gereksiz yere dolmasına ve performansın düşmesine neden olur. Uzun süre devam ettiğinde, uygulama yavaşlar veya çökebilir.

Sorunun Çözümü

Bellek sızıntılarını çözmek için kodda bellek yönetimi dikkatle izlenmelidir. Çöp toplayıcılar (garbage collectors) gibi araçlar kullanarak gereksiz veriler silinebilir. Ayrıca, bellek analizi araçları ile sızıntı yapan bileşenler tespit edilip hızlıca müdahale edilebilir. BoundsChecker, Deleaker, Memory Validator, IBM Rasyonel Purify



Bellek Sızıntısı

Güvenlik Açıklı Kod

```c
void processUserInput(const char *input) {

    char *buffer = (char *)malloc(256);

    // Girdiyi kopyalıyoruz
    strcpy(buffer, input);

    // Girdinin geçerli olup olmadığını kontrol ediyoruz
    if (strlen(input) > 100) {
        printf("Girdi çok uzun, işlenemiyor.\n");
        return; // Belleği serbest bırakmadan çıkış
    }

    printf("Girdi: %s\n", buffer);

    free(buffer);
}

void main() {
    char userInput[300];

    printf("Bir metin girin: ");
    fgets(userInput, sizeof(userInput), stdin);

    processUserInput(userInput);
}
```

Güvenli Kod

```c
void processUserInput(const char *input) {

    char *buffer = (char *)malloc(256);

    // Girdiyi kopyalıyoruz
    strcpy(buffer, input);

    // Girdinin geçerli olup olmadığını kontrol ediyoruz
    if (strlen(input) > 100) {
        printf("Girdi çok uzun, işlenemiyor.\n");
        free(buffer);
        return;
    }

    // Girdiyi işliyoruz
    printf("Girdi: %s\n", buffer);

    // Belleği serbest bırakıyoruz
    free(buffer);
}

int void() {
    char userInput[300];

    printf("Bir metin girin: ");
    fgets(userInput, sizeof(userInput), stdin);

    processUserInput(userInput);
}
```

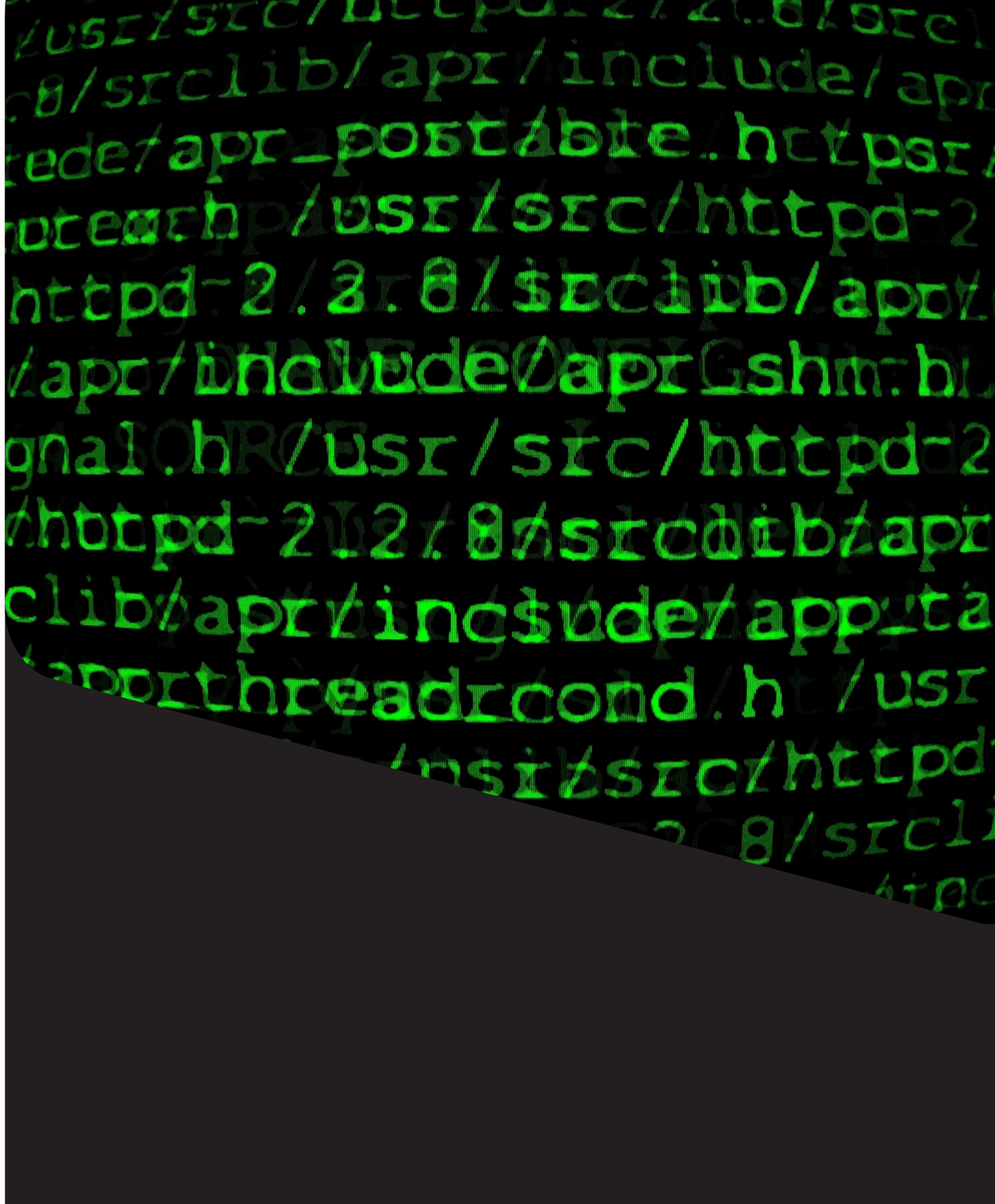
Fragmentasyon

Sorunun Tanımı

Fragmentasyon, bir dosyanın veya verinin bellekte dağınık şekilde depolanması durumudur. Bu dağınıklık, dosyaların parçalanarak farklı bellek alanlarına dağılmasına neden olur ve sistem performansını düşürebilir. Zamanla, dosya işlemleri daha yavaş hale gelir ve bellek verimli kullanılamaz.

Sorunun Çözümü

Fragmentasyonu önlemek için düzenli olarak disk birleştirme işlemi yapılabilir. Bu işlem, dağınık dosya parçalarını bir araya getirerek daha verimli bir bellek yapısı sağlar. Ek olarak, modern işletim sistemlerinde otomatik birleştirme araçları kullanılarak performans kaybı en aza indirilebilir.



Fragmentasyon

Güvenlik Açıklı Kod

```
void main() {
    // Bellek tahsisi
    int* block1 = (int*) malloc(5 * sizeof(int));  // 5 tamsayı boyutunda bellek tahsisi
    int* block2 = (int*) malloc(10 * sizeof(int)); // 10 tamsayı boyutunda bellek tahsisi

    free(block1);

    int* block3 = (int*) malloc(8 * sizeof(int));  // 8 tamsayı boyutunda bellek tahsisi

    // Eğer 8 elemanlık bellek tahsisi yapılabiliyorsa
    if (block3 != nullptr) {
        std::cout << "Yeni büyük bellek bloğu (8 eleman) başarıyla tahsis edildi!" << std::endl;
    } else {
        std::cout << "Yeterli bellek bulunamadı!" << std::endl;
    }

    free(block2);
    free(block3);

}
```

Güvenli Kod için **MemoryPool** Kullanılabilir.

Öncesi

BELLEK
5 Tamsayı Boyutu
10 Tamsayı Boyutu

Sonrası

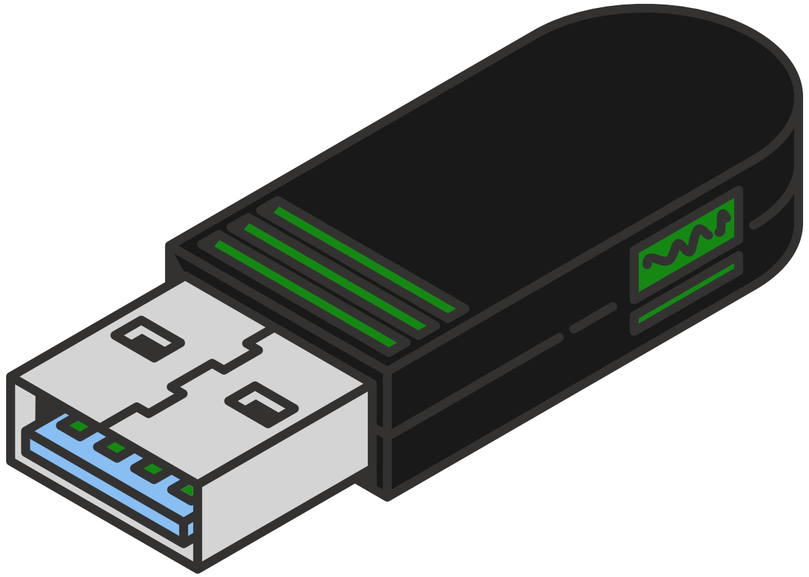
BELLEK
Yeni 5 Tamsayı Boyutu
10 Tamsayı Boyutu
Yeni 3 Tamsayı Boyutu

Bellek Çakışması (Memory Corruption)

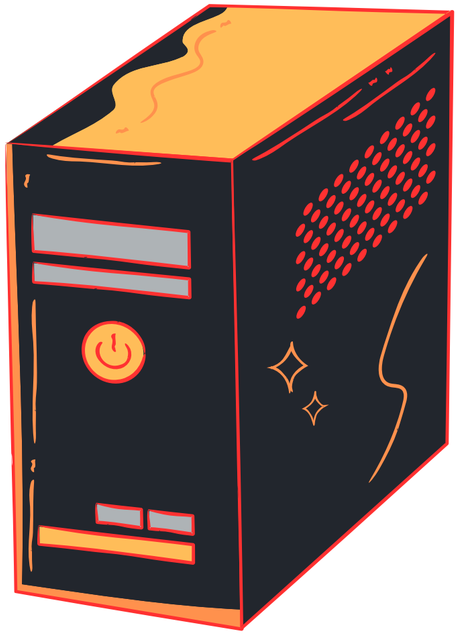
Programlar, bellek alanlarını sınırsız şekilde kullanmaya çalıştığında bu tür sorunlar ortaya çıkar. Buffer overflow, bir tampon (buffer) üzerinde yapılan bir yazma işleminin tampon sınırını aşması ve ardındaki bellek alanına zarar vermesi durumudur. Bu tür hatalar, bellek çakışmalarına ve veri bozulmasına yol açabilir.

Sorunun Çözümü

- Gelişmiş bellek yönetim teknikleri kullanarak buffer overflow saldırılarını engellemeye yardımcı olun.
- Statik ve dinamik analiz araçları ile kodunuzu düzenli olarak kontrol edin.
- Kod incelemeleri ve testlerle yazılımınızı denetleyin.



Yavaş Bellek Erişimi (Memory Access Latency)



Bellek erişim gecikmesi, bir işlemcinin belleği okuma veya yazma işleminde yaşadığı gecikmelerdir. Bu tür gecikmeler, sistemin genel performansını ciddi şekilde etkileyebilir çünkü bellek erişimi işlemcinin veri işleme hızını doğrudan etkiler. Bellek erişim süreleri genellikle latency (gecikme) olarak ölçülür ve bu gecikmeler, bellek hiyerarşisindeki farklı seviyelere (L1, L2, L3 önbellekler, ana bellek, disk) bağlı olarak değişir.

Sorunun Çözümü

İşlemciler, L1, L2, L3 önbellekleri kullanarak veriye daha hızlı erişim sağlarlar. Verinin önbelleğe yerleştirilmesi ve gerektiğinde işleme alınması, bellek erişim hızını artırabilir. Bu nedenle, yazılımlar, daha fazla temporal locality (zaman yakınlığı) ve spatial locality (mekansal yakınlık) sağlamaya yönelik yazılmalıdır.

Önbellek sıralaması (cache alignment) ve veri dizileri (data blocking) gibi optimizasyon teknikleri, verinin önbellek hit oranını artırabilir ve bu da bellek erişim hızını iyileştirir.

Kaynakça

- https://en.wikipedia.org/wiki/Memory_leak
- https://www.geeksforgeeks.org/what-is-memory-leak-how-can-we-avoid/
- https://www.edn.com/handling-memory-fragmentation/
- https://en.wikipedia.org/wiki/Fragmentation_(computing)
- https://en.wikipedia.org/wiki/Segmentation_fault
- https://akcayonur.medium.com/buffer-overflow-zafiyeti-nedir
- https://www.ibm.com/docs/tr/aix/7.3?topic=performance-hardware-hierarchy